DATABASE TECHNOLOGIES LAB SHEET – 2

Course Code: MCA105P

SQL LAB SHEET

Section A: Easy Level (Q1–Q20)

Q1. Basic Data Insertion

Question: Insert a new employee 'Fiona Gallagher' into the employees table with ID 106, dept_id 10, salary 65000.00, hire_date '2024-02-01', and is_active TRUE.

SQL Syntax:

```
INSERT INTO employees
(emp_id, first_name, last_name, dept_id, salary, hire_date, is_active)
VALUES
(106, 'Fiona', 'Gallagher', 10, 65000.00, '2024-02-01', TRUE);
```

Q2. Basic Data Insertion

Question: Insert a new department 'Quality Assurance' located in 'Building B' with a budget of 250000.00 into departments, setting dept_id to 60.

SQL Syntax:

```
INSERT INTO departments
(dept_id, dept_name, location, budget)
VALUES
(60, 'Quality Assurance', 'Building B', 250000.00);
```

Q3. Basic Data Insertion

Question: Insert a new order for customer 'Delta Retail' dated '2024-01-28' worth 2100.00 with status 'Pending' into orders.

SQL Syntax:

```
INSERT INTO orders
(order_id, customer_name, order_date, total_amount, status)
VALUES
(5006, 'Delta Retail', '2024-01-28', 2100.00, 'Pending');
```

Q4. Data Retrieval

Question: Retrieve all columns from the employees table.

SQL Syntax:

```
SELECT *
FROM employees;
```

Q5. Data Retrieval

Question: Retrieve only the first_name, last_name, and salary of all employees.

SQL Syntax:

```
SELECT first_name, last_name, salary
FROM employees;
```

Q6. Data Retrieval and Filtering

Question: Find all employees who belong to department 10.

SQL Syntax:

```
SELECT *
FROM employees
WHERE dept_id = 10;
```

Q7. Data Retrieval and Filtering

Question: Display all orders where the status is 'Completed'.

SQL Syntax:

```
SELECT *
FROM orders
WHERE status = 'Completed';
```

Q8. Updating Records

Question: Update the salary of employee with emp_id 102 to 55000.00.

SQL Syntax:

```
UPDATE employees
SET salary = 55000.00
WHERE emp_id = 102;
```

Q9. Updating Records

Question: Change the status of order 5002 from 'Pending' to 'Shipped'.

SQL Syntax:

```
UPDATE orders
SET status = 'Shipped'
WHERE order_id = 5002
```

```
AND status = 'Pending';
```

Q10. Updating Records

Question: Increase the budget of department 20 by 20000.00.

SQL Syntax:

```
UPDATE departments
SET budget = budget + 20000.00
WHERE dept_id = 20;
```

Q11. Deleting Records

Question: Delete the order record with order_id 5004 from the orders table.

SQL Syntax:

```
DELETE FROM orders
WHERE order_id = 5004;
```

Q12. Deleting Records

Question: Remove employee 'Diana Prince' (emp_id 104) from the employees table.

SQL Syntax:

```
DELETE FROM employees
WHERE emp_id = 104
AND first_name = 'Diana'
AND last_name = 'Prince';
```

Q13. Condition Filtering

Question: Find all employees with a salary greater than 60000.00.

SQL Syntax:

```
SELECT *
FROM employees
WHERE salary > 60000.00;
```

Q14. Range Filtering

Question: Retrieve all orders placed between '2024-01-15' and '2024-01-20'.

SQL Syntax:

```
SELECT *
FROM orders
WHERE order_date BETWEEN '2024-01-15' AND '2024-01-20';
```

Q15. Set Filtering

Question: Display all departments located in either 'Building A' or 'Building B'.

SQL Syntax:

```
SELECT *
FROM departments
WHERE location IN ('Building A', 'Building B');
```

Q16. Pattern Matching

Question: Find all employees whose last_name starts with the letter 'S'.

SQL Syntax:

```
SELECT *
FROM employees
WHERE last_name LIKE 'S%';
```

Q17. Sorting Results

Question: List all employees sorted by salary in descending order.

SQL Syntax:

```
SELECT *
FROM employees
ORDER BY salary DESC;
```

Q18. Sorting Results

Question: Retrieve all departments sorted alphabetically by department name.

SQL Syntax:

```
SELECT *
FROM departments
ORDER BY dept_name ASC;
```

Q19. Basic Aggregation

Question: Count the total number of records in the employees table.

SQL Syntax:

```
SELECT COUNT(*) AS total_employees
FROM employees;
```

Q20. Basic Aggregation

Question: Calculate the average total_amount of all orders.

SQL Syntax:

```
SELECT AVG(total_amount) AS average_order_amount
FROM orders;
```

Section B: Medium Level (Q21–Q40)

Q21. Multi-Row Insert

Question: Insert two new employees ('George Clark', dept_id 30, salary 50000.00, hire_date '2024-02-05', TRUE) and ('Hannah Abbott', dept_id 10, salary 71000.00, hire_date '2024-02-10', TRUE) into the employees table in a single query.

SQL Syntax:

```
INSERT INTO employees
(emp_id, first_name, last_name, dept_id, salary, hire_date, is_active)
VALUES
(107, 'George', 'Clark', 30, 50000.00, '2024-02-05', TRUE),
(108, 'Hannah', 'Abbott', 10, 71000.00, '2024-02-10', TRUE);
```

Q22. Complex Updates

Question: Give a 10% salary raise to all employees working in department 10.

SQL Syntax:

```
UPDATE employees
SET salary = salary * 1.10
WHERE dept_id = 10;
```

Q23. Complex Updates

Question: Deactivate (set is_active = FALSE) all employees hired before '2021-01-01'.

SQL Syntax:

```
UPDATE employees
SET is_active = FALSE
WHERE hire_date < '2021-01-01';
```

Q24. Multi-Condition Filtering

Question: Retrieve all active employees (is_active = TRUE) who earn more than 60000.00.

SQL Syntax:

```
SELECT *
FROM employees
WHERE is_active = TRUE
AND salary > 60000.00;
```

Q25. Multi-Condition Filtering

Question: Find orders that are either 'Cancelled' OR have a total_amount greater than 2000.00.

SQL Syntax:

```
SELECT *
FROM orders
WHERE status = 'Cancelled'
OR total_amount > 2000.00;
```

Q26. Set Exclusion

Question: List all employees who do NOT belong to departments 10 or 30.

SQL Syntax:

```
SELECT *
FROM employees
WHERE dept_id NOT IN (10, 30);
```

Q27. Pattern Matching

Question: Find all employees whose first_name contains the letter 'a' (case-insensitive).

SQL Syntax:

```
SELECT *
FROM employees
WHERE LOWER(first_name) LIKE '%a%';
```

Q28. Grouping and Aggregation

Question: Calculate total salary expenditure for each department.

SQL Syntax:

```
SELECT dept_id, SUM(salary) AS total_salary
FROM employees
GROUP BY dept_id;
```

Q29. Grouping and Aggregation

Question: Find the total number of employees in each department.

SQL Syntax:

```
SELECT dept_id, COUNT(*) AS total_employees
FROM employees
GROUP BY dept_id;
```

Q30. Group Filtering (HAVING)

Question: Display departments that have a total salary expense exceeding 100000.00.

SQL Syntax:

```
SELECT dept_id, SUM(salary) AS total_salary
FROM employees
GROUP BY dept_id
HAVING SUM(salary) > 100000.00;
```

Q31. Grouping and Aggregation

Question: Find maximum and minimum order amounts per order status.

SQL Syntax:

```
SELECT status,
       MAX(total_amount) AS maximum_amount,
       MIN(total_amount) AS minimum_amount
FROM orders
GROUP BY status;
```

Q32. Subqueries

Question: Retrieve all employees who earn more than the average employee salary.

SQL Syntax:

```
SELECT *
FROM employees
WHERE salary > (
    SELECT AVG(salary)
    FROM employees
);
```

Q33. Subqueries

Question: Find all employees who work in departments located in 'Building A'.

SQL Syntax:

```
SELECT *
FROM employees
WHERE dept_id IN (
    SELECT dept_id
    FROM departments
    WHERE location = 'Building A'
);
```

Q34. Nested Deletions

Question: Delete all orders placed by customers who have an order with 'Cancelled' status.

SQL Syntax:

```
DELETE FROM orders
WHERE customer_name IN (
    SELECT customer_name
    FROM (
        SELECT customer_name
        FROM orders
        WHERE status = 'Cancelled'
    ) AS cancelled_customers
);
```

Q35. NULL Handling

Question: Find all employees whose last_name is NULL (if any exists).

SQL Syntax:

```
SELECT *
FROM employees
WHERE last_name IS NULL;
```

Q36. String Functions

Question: Display full names of employees by concatenating first_name and last_name with a space.

SQL Syntax:

```
SELECT CONCAT(first_name, ' ', last_name) AS full_name
FROM employees;
```

Q37. Ordering and Limits

Question: Retrieve the top 3 highest-paid employees.

SQL Syntax:

```
SELECT *
FROM employees
ORDER BY salary DESC
LIMIT 3;
```

Q38. Multi-Column Ordering

Question: List employees sorted by department ID ascending, then by salary descending.

SQL Syntax:

```
SELECT *
FROM employees
ORDER BY dept_id ASC, salary DESC;
```

Q39. Complex Deletions

Question: Delete all departments that have a budget less than 200000.00.

SQL Syntax:

```
DELETE FROM departments
WHERE budget < 200000.00;
```

Note: If foreign-key constraints exist, deleting a department referenced by employees may fail.

Q40. Bulk Updates

Question: Update all orders with amount under 200.00 to have status 'Cancelled'.

SQL Syntax:

```
UPDATE orders
SET status = 'Cancelled'
WHERE total_amount < 200.00;
```

Section C: Hard / Advanced (Q41–Q50)

Q41. Advanced Subqueries and DML

Question: Increase salary by 15% for employees working in the department with the highest budget.

SQL Syntax:

```
UPDATE employees
SET salary = salary * 1.15
WHERE dept_id = (
    SELECT dept_id
    FROM departments
    WHERE budget = (
        SELECT MAX(budget)
        FROM departments
    )
);
```

Q42. Group Aggregations and Subqueries

Question: Find departments where average employee salary is greater than the global average employee salary.

SQL Syntax:

```
SELECT dept_id, AVG(salary) AS average_salary
FROM employees
```

```
GROUP BY dept_id
HAVING AVG(salary) > (
    SELECT AVG(salary)
    FROM employees
);
```

Q43. Relational Subqueries

Question: Display employee details for employees who belong to departments with a budget greater than 300000.00.

SQL Syntax:

```
SELECT *
FROM employees
WHERE dept_id IN (
    SELECT dept_id
    FROM departments
    WHERE budget > 300000.00
);
```

Q44. Offset and Ranking

Question: Find the third-highest salary in the employees table using subqueries or OFFSET.

SQL Syntax:

```
SELECT DISTINCT salary
FROM employees
ORDER BY salary DESC
LIMIT 1 OFFSET 2;
```

Q45. Conditional Updates (CASE)

Question: Update employees: Increase salary by 10% if salary is less than 60000, otherwise increase by 5% using a CASE expression.

SQL Syntax:

```
UPDATE employees
SET salary = CASE
    WHEN salary < 60000 THEN salary * 1.10
    ELSE salary * 1.05
END;
```

Q46. Correlated Subqueries and DML

Question: Delete employees who are inactive (is_active = FALSE) and earn less than their department's average salary.

SQL Syntax:

```
DELETE FROM employees
WHERE is_active = FALSE
```

```
AND salary < (
    SELECT avg_salary
    FROM (
        SELECT dept_id, AVG(salary) AS avg_salary
        FROM employees
        GROUP BY dept_id
    ) AS department_averages
    WHERE department_averages.dept_id = employees.dept_id
);
```

Q47. Scalar Subqueries in Projection

Question: Find department name and total employee count using a scalar subquery in the SELECT projection list.

SQL Syntax:

```
SELECT
    d.dept_name,
    (
        SELECT COUNT(*)
        FROM employees e
        WHERE e.dept_id = d.dept_id
    ) AS total_employees
FROM departments d;
```

Q48. Complex Group Filtering

Question: List customer names who have placed more than one order OR have a total spending sum greater than 1000.00.

SQL Syntax:

```
SELECT customer_name
FROM orders
GROUP BY customer_name
HAVING COUNT(*) > 1
OR SUM(total_amount) > 1000.00;
```

Q49. Data Transformation and Copying

Question: Insert into employees a copy of all employees from department 10 into department 50 with modified IDs (+500).

SQL Syntax:

```
INSERT INTO employees
(emp_id, first_name, last_name, dept_id, salary, hire_date, is_active)
SELECT
    emp_id + 500,
    first_name,
    last_name,
    50,
    salary,
    hire_date,
    is_active
```

```
FROM employees
WHERE dept_id = 10;
```

Q50. Multi-Branch CASE Updates

Question: Bulk update order statuses: 'Completed' if total_amount is greater than 1000, 'Shipped' if between 500 and 1000, otherwise 'Pending'.

SQL Syntax:

```
UPDATE orders
SET status = CASE
    WHEN total_amount > 1000 THEN 'Completed'
    WHEN total_amount BETWEEN 500 AND 1000 THEN 'Shipped'
    ELSE 'Pending'
END;
```